

Dataset Transformation and Validation Pipeline Report

Sathvika Ayyappa Prabhu

June 17, 2025

1. Introduction

This report provides a comprehensive overview of the dataset transformation and validation pipeline developed to process the **AGUVIS** and **OS-Atlas** datasets. The pipeline focuses on ensuring data integrity by performing rigorous validation checks, handling edge cases, and generating detailed logs for tracking parsing progress, warnings, and errors. The validation process ensures that any dataset processed meets the required standards and that issues, whether critical or minor, are effectively flagged.

2. Project Flow and Structure

The pipeline operates through a clear and structured flow, ensuring datasets are parsed, validated, and logged effectively:

1. Dataset Configuration (YAML)

The process begins with a **YAML configuration file** that outlines the paths to dataset files (both images and JSON). This configuration allows the pipeline to be flexible and handle various datasets dynamically, making it adaptable to other future datasets as well.

2. Parsing

Datasets are parsed by reading JSON and image files. Each entry is processed, applying necessary transformations such as bounding box normalization (for **OS-Atlas**) and validation checks.

3. Validation and Error Handling

- **For OS-Atlas:** Includes bounding box normalization, image path checks, and UI element validation.
- **For AGUVIS:** Focuses on validating `pyautogui` commands, checking proper alternation between human and GPT turns, and ensuring completeness of conversation data.

4. Logging

All parsing results are logged into separate directories:

- **Skipped Elements:** Files or elements that failed critical validation checks and cannot be processed.
- **Warnings:** Files or elements with minor issues that can still be processed.
- **Failure Logs:** Tracks datasets that encountered parsing errors.
- **Summary Logs:** Provides an overall summary of the parsing process, showing the number of successfully parsed, skipped, and warned elements.

Example Summary Log:

```
{
  "linux_splited.json": {
    "total": 9671,
    "parsed": 9664,
    "skipped": 7,
    "warnings": 0,
    "parsed_percent": 99.93,
    "skipped_percent": 0.07,
    "warning_percent": 0.0
  }
}
```

3. Dataset Parsing Details

AGUVIS Dataset Parsing

For the **AGUVIS** dataset, parsing and validation processes were handled as follows:

- **Valid pyautogui Commands:** The pipeline verifies the correctness of pyautogui commands (such as `pyautogui.click` or `pyautogui.moveTo`). Invalid commands—such as those with incorrect syntax or unsupported functions—are flagged as **critical issues** and skipped because invalid commands could lead to failed UI automation or unpredictable behavior. This ensures that any subsequent data processing steps based on pyautogui commands remain valid and actionable.
- **Human-to-GPT Conversation Flow:** The pipeline validates that **human** and **GPT** turns alternate properly. Any deviation, such as a human turn not being followed by a GPT turn or vice versa, results in a **warning**.
- **Handling of Stage-Specific Data:** In **Stage 1** of the conversation, certain fields, such as `text_observation_desc`, `text_reasoning`, and `text_subtask`, are not expected to appear. These fields are generally reserved for **Stage 2** interactions, and their presence in **Stage 1** triggers a **warning**.
- **Target Point Validation:** If a `target_point` is used within a `UIAction`, its coordinates are validated to ensure they fall within the image’s dimensions. **Critical issues** are raised if the coordinates are outside the image bounds.

OS-Atlas Dataset Parsing

For **OS-Atlas**, the pipeline performed:

- **Bounding Box Normalization:** Each entry’s bounding box was converted from relative coordinates to absolute pixel values based on the image’s dimensions.
- **Image Path Validation:** The pipeline checks that each image path is correctly formatted, non-empty, and relative (not absolute).
- **UI Element Validation:** Ensured that each UI element had both a valid `element_type` and a non-empty `instruction`.

4. Validation Rules and Edge Case Handling

The system applied comprehensive validation rules to ensure that the data adhered to the required quality standards.

AGUVIS-Specific Validation

- **Valid pyautogui Commands:** Ensured pyautogui commands are valid, skipping any invalid ones.
- **Human-to-GPT Alternation:** Checked that each human input was followed by a GPT output, and vice versa. Warnings were issued if the alternation was broken.
- **Stage-Specific Field Validation:** Ensured that Stage 1 and Stage 2 fields were used correctly according to the stage of the conversation.
- **Target Point Validation:** Ensured that any `target_point` coordinates were within the image's bounds.

OS-Atlas-Specific Validation

- **Bounding Box Normalization:** Ensured bounding boxes were normalized and within the image's dimensions.
- **Image Path Validation:** Checked image paths to ensure they were valid and relative.
- **UI Element Validation:** Ensured each UI element had a valid `element_type` and non-empty `instruction`.

5. Reasons for Skipping and Warnings

Skipped Elements

- **Critical Issues:** Invalid pyautogui actions, out-of-bounds bounding boxes, and missing critical fields (e.g., `element_type` or `instruction`) were skipped because they rendered the element unusable for further processing.

Warnings

- **Minor Issues:** Elements with minor issues, such as small bounding boxes, invalid or missing instructions, and broken human-to-GPT conversation flow, were flagged with warnings but were not skipped.

6. Dataset Summary and Performance

The following table summarizes the parsing results for all datasets:

```
{
  "total_samples": 53847,
  "datasets": {
    "linux_splited.json": {
      "total": 9671,
      "parsed": 9664,
      "skipped": 7,
      "warnings": 0,
      "parsed_percent": 99.93,
      "skipped_percent": 0.07,
      "warning_percent": 0.0
    },
    "macos_splited.json": {
      "total": 4441,
      "parsed": 4440,
      "skipped": 1,
      "warnings": 0,
      "parsed_percent": 99.98,
      "skipped_percent": 0.02,
      "warning_percent": 0.0
    },
    "ricosca.json": {
      "total": 18148,
      "parsed": 18146,
      "skipped": 2,
      "warnings": 0,
      "parsed_percent": 99.99,
      "skipped_percent": 0.01,
      "warning_percent": 0.0
    },
    "uibert_raw.json": {
      "total": 5682,
      "parsed": 5682,
      "skipped": 0,
      "warnings": 0,
      "parsed_percent": 100.0,
      "skipped_percent": 0.0,

```

```

        "warning_percent": 0.0
    },
    "aw_mobile.json": {
        "total": 15905,
        "parsed": 15869,
        "skipped": 36,
        "warnings": 0,
        "parsed_percent": 99.77,
        "skipped_percent": 0.23,
        "warning_percent": 0.0
    }
}

```

7. Limitations and Future Work

WebArena Dataset

The parsing for the **WebArena** dataset was delayed due to the lengthy process of collecting demonstrations and converting pickle data to JSON format. However, this dataset will significantly enhance the pipeline once fully integrated.

Impact of Combining Datasets

Integrating **AGUVIS**, **OS-Atlas**, and **WebArena** will result in a highly versatile pipeline capable of processing a wide variety of UI interaction data, significantly enhancing its robustness and analytical capability.

8. Conclusion

The dataset transformation and validation pipeline successfully processes the **AGUVIS** and **OS-Atlas** datasets, ensuring high-quality data through detailed validation and error handling. The integration of **WebArena** and other datasets will further enhance the pipeline's capabilities, providing a robust solution for UI interaction data processing and analysis.